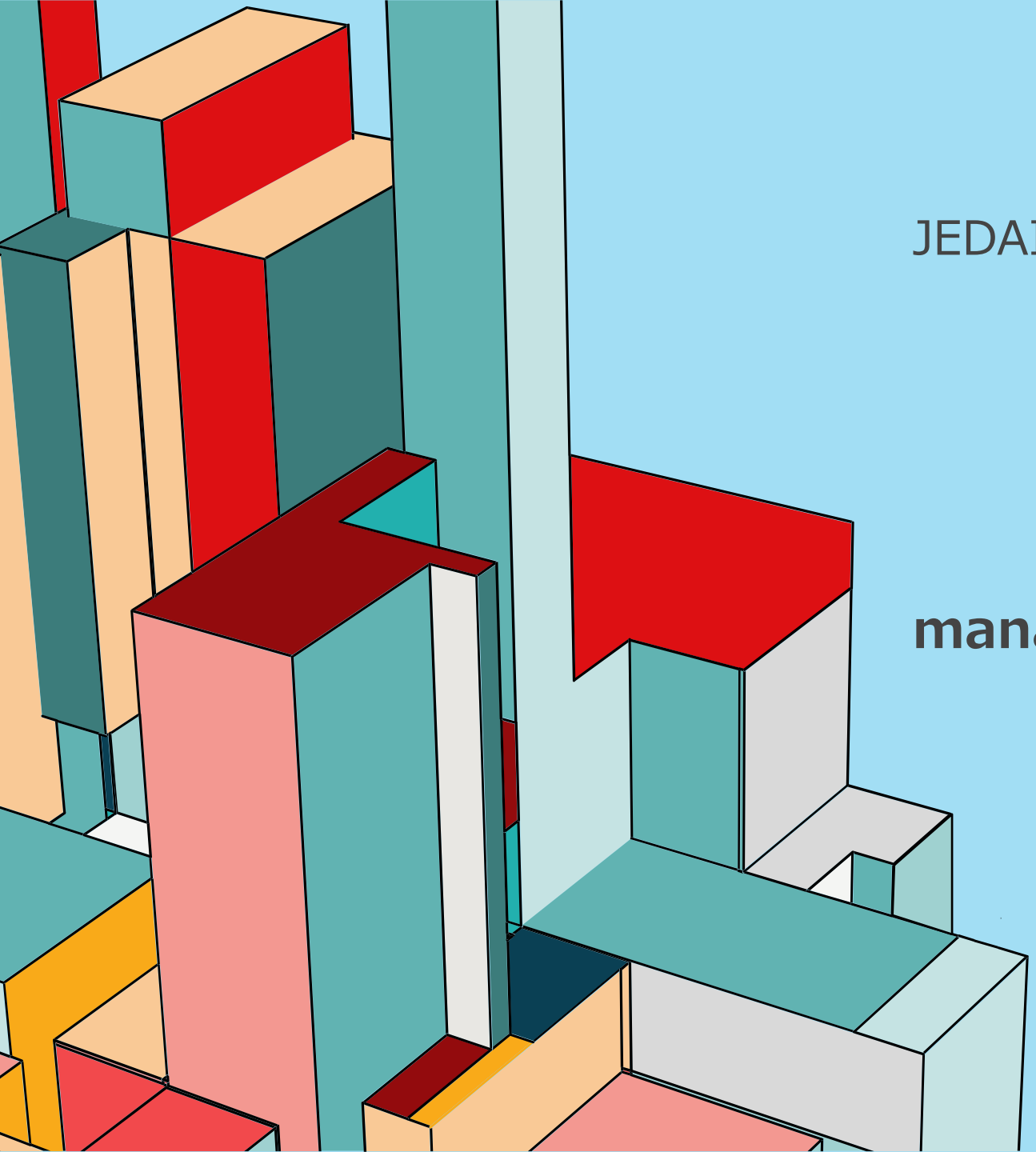


JEDAI in Osaka 2026 春

生成AIによる Databricksの開発方法論を 改めて考えてみた

manabian

はじめに

- 本発表では、生成AIの開発手法（Skillsなど）に関する最新情報の共有ではなく、Databricksを活用したシステム開発に役立てるための心構え（ほぼ決意）をお伝えします。

- データ分析におけるコードは少量で複雑かつ分割が難しい場合が多く、他のプログラミング技術をそのまま応用できないケースがあります。

- 私の業務上、利用環境を固定できないことが多かったものの、現在は整った環境下でデータ分析システムのライフサイクルにおける生成AI活用の検証を進めています。今後は、その観点からの情報発信も行ってまいります。

自己紹介

@manabian

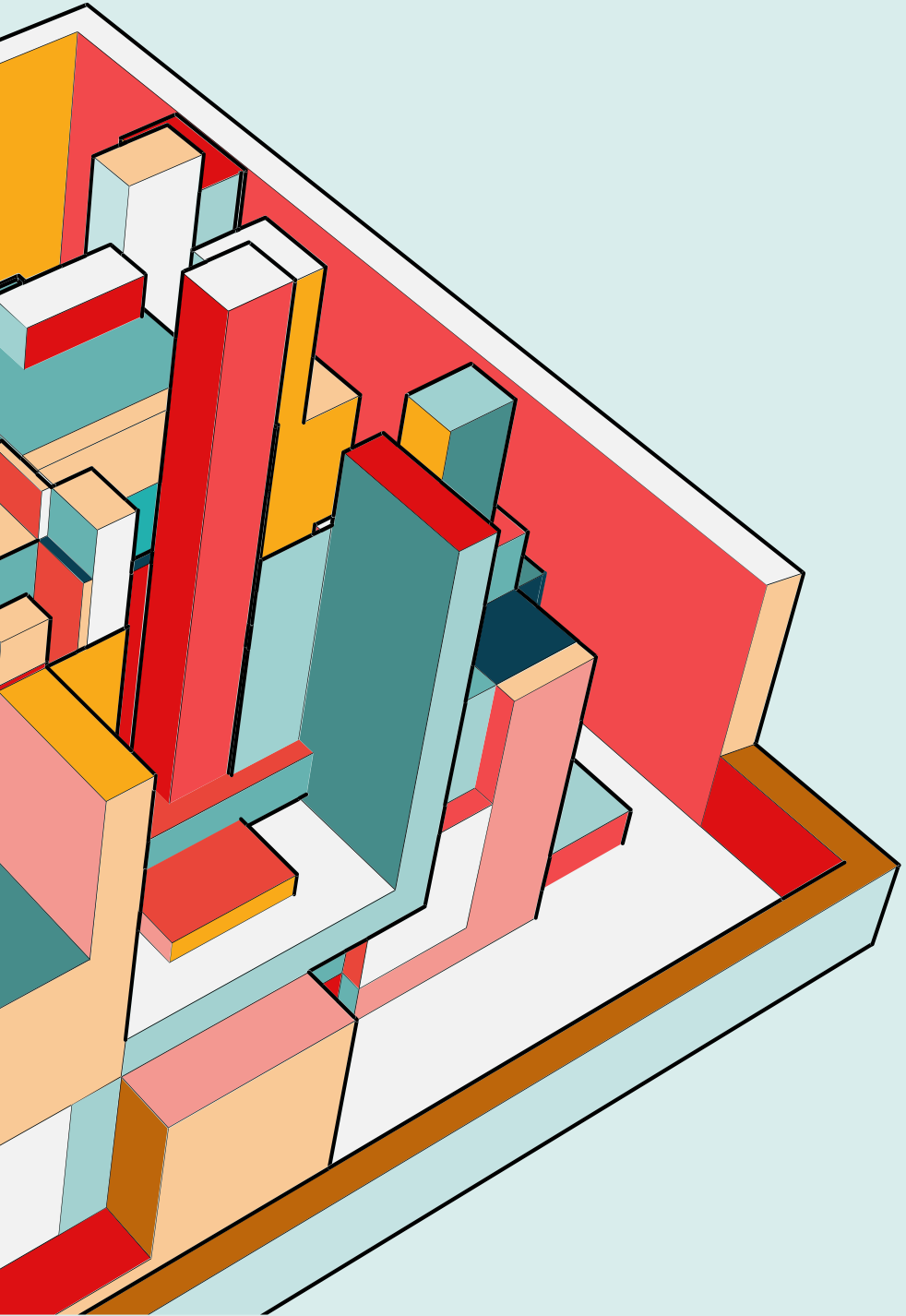
【主業務】

レイクハウス・DWH・BI構築に関するSIベンダーにてフェローとして、次の業務を実施。

- データエンジニアリングの実装論の整理とライブラリ開発
- データ分析基盤とアプリケーションの設計・構築に関する支援
- データ分析基盤に関する技術検証

【趣味】

趣味がデータ分析基盤に関する調査や検証であり、
主に Qiita を中心に データ分析基盤プラットフォームに関する記事を投稿。



生成 AI による開発論の現状

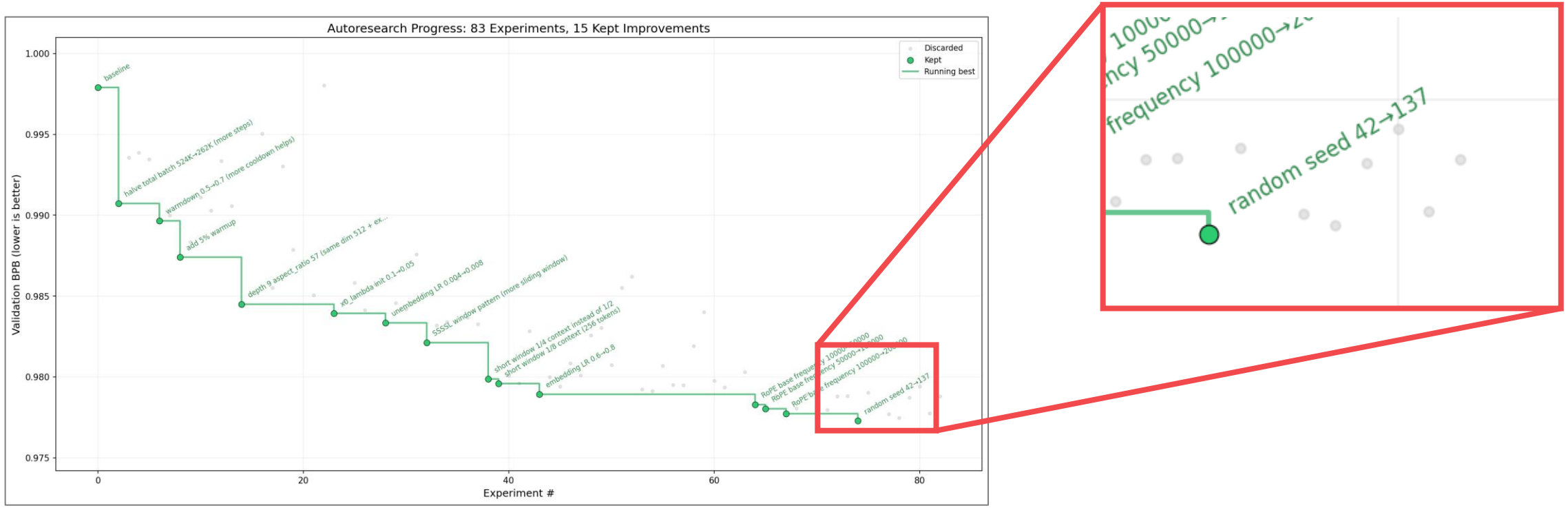
生成 AI の技術力は Copilot からホームズへ

- 生成AIによる開発は当初Copilot（副操縦士）から始まりました。
- 現在では、アウトプットの品質が飛躍的に向上し、
  私たちの多くの課題を解決してくれる名探偵ホームズのような存在へと進化しています。

GitHub Copilot

生成 AI のアウトプットの過信は禁物

- AIエージェントを活用した継続的な機械学習モデルの精度向上を目指す autoresearch が最近注目されました。
- その施策の一つに seedの調整といった裏技的な手法も用いられており、現段階では完全に自律して運用できる状況には至っていないと考えています。



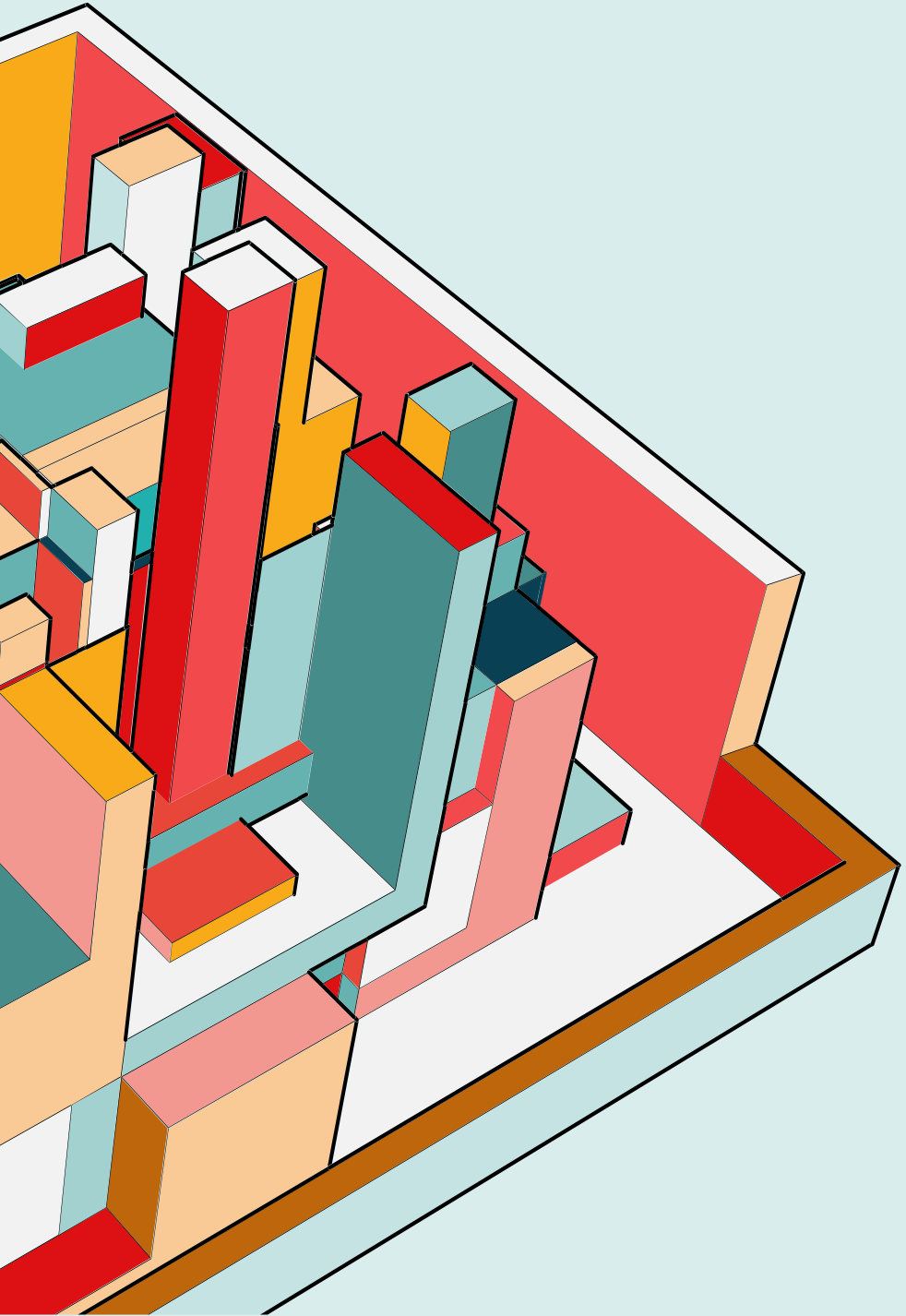
引用元：[GitHub - karpathy/autoresearch: AI agents running research on single-GPU nanochat training automatically · GitHub]

生成 AI はホームズ、私たちはワトソンという関係が理想

- 名探偵ホームズの助手であるワトソンの役割を担い、私たちは情報（コンテキスト）を収集し、その重要度を判断してストーリー（システム）を構築する必要があります。

ホームズを例にした理由

非公開画像



名探偵と良好な関係を築ける
ワトソンになる方法

<問題> 生成 AI の出力を鵜呑みにして量産すること (1/2)

- 生成 AI の言いなりとなりコードを大量生産開を実施するケースがありますが、
  ライブラリ化や回帰テスト等により品質をコントロールしたプログラムの開発が必要です。

コードの大量生産

データを
UPSERT で
連携する SQL を
記述して

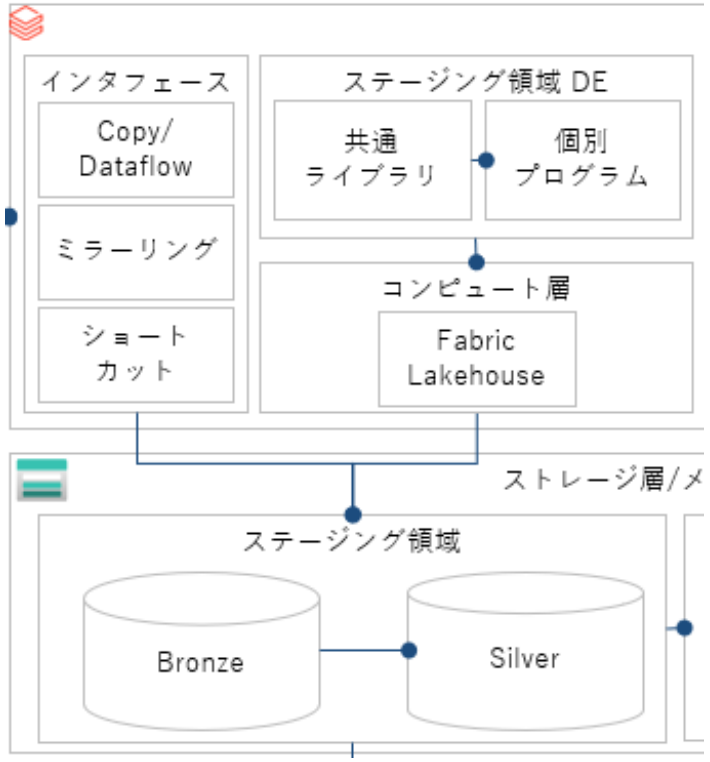
データを
UPSERT で
連携する SQL を
記述して

FactLineItem_UPSERT.sql

sample_sqls > facts > FactLineItem_UPSERT.sql

```sql
MERGE INTO lineitem AS target
USING (
    SELECT
        lineitem_id,
        order_id,
        product_id,
        order_quantity,
        unit_price,
        extended_amount,
        discount_amount,
        line_total,
        carrier_tracking_number,
        modified_date,
        current_timestamp() AS updated_at
    FROM source_data  -- ソーステーブル/ビュー名に変更
    WHERE modified_date >= CURRENT_DATE() - INTERVAL '7 days'  -- 必要に応じて条件変更
) AS source
ON target.lineitem_id = source.lineitem_id
  AND target.order_id = source.order_id
WHEN MATCHED THEN UPDATE SET
    product_id = source.product_id,
    order_quantity = source.order_quantity,
    unit_price = source.unit_price,
    extended_amount = source.extended_amount,
    discount_amount = source.discount_amount,
    line_total = source.line_total,
    carrier_tracking_number = source.carrier_tracking_number,
    modified_date = source.modified_date,
    updated_at = source.updated_at
WHEN NOT MATCHED THEN INSERT
    (lineitem_id, order_id, product_id, order_quantity, unit_price,
     extended_amount, discount_amount, line_total, carrier_tracking_number,
     modified_date, updated_at)
    VALUES
    (source.lineitem_id, source.order_id, source.product_id, source.order_quantity, source.unit_
     source.extended_amount, source.discount_amount, source.line_total, source.carrier_tracking_
     source.modified_date, source.updated_at);
```

共通化



<問題> 生成 AI の出力を鵜呑みにして量産すること (2/2)

- 宣言型によりデータエンジニアリングを実施できる機能を利用することも品質のばらつきを抑えるためにはとても有効な選択肢です。
- 自作プログラムは柔軟だが、再利用・標準化・テスト・保守・参入障壁に課題が出てきたという事例 *1 もあります。

SDP

Lakeflow Spark宣言型パイプライン

Lakeflow Spark宣言型パイプライン (SDP) は、 SQLおよびPythonでバッチおよびストリーミング データ パイプラインを作成するためのフレームワークです。 Lakeflow SDP は、パフォーマンスが最適化されたDatabricks Runtime上で実行されながら、 Apache Spark 宣言型パイプラインを拡張し、相互運用可能です。 パイプラインの一般的な使用例には、クラウド ストレージ (Amazon S3 、 Azure ADLS Gen2、 Google Cloud Storageなど) やメッセージ バス (Apache Kafka 、 Amazon Kinesis 、Google Pub/Sub、 Azure EventHub、 Apache Pulsar など) などのソースからのデータ取り込み、増分バッチおよびストリーミング変換が含まれます。

このセクションでは、パイプラインの使用に関する詳細情報を提供します。次のトピックは、開始する際に役立ちます。

引用元：Lakeflow Spark宣言型パイプライン | Databricks on AWS

dbt

dbt Coreへの接続

このページでは、dbt とは何か、dbt Core のインストール方法、接続方法について説明します。dbt Cloud と呼ばれる dbt のホスト バージョンも利用可能です。詳細については、 dbt Cloudへの接続」を参照してください。

dbtとは何ですか?

dbt (Data Build Tool) は、select ステートメントを記述してデータを変換するための開発環境です。 dbt は、これらの SELECT 文をテーブルとビューに変換します。 dbt は、コードを生の SQL にコンパイルし、Databricks の指定されたデータベースでそのコードを実行します。 dbt は、バージョン管理、ドキュメント化、モジュール性など、協調的なコーディング パターンとベスト プラクティスをサポートします。

引用元：GitHub - karpathy/autoresearch: AI agents running research on single-GPU nanochat training automatically · GitHub

*1 Leveling up data engineering at Riot: How they transformed DevEx with dbt (Coalesce 2025) - YouTube

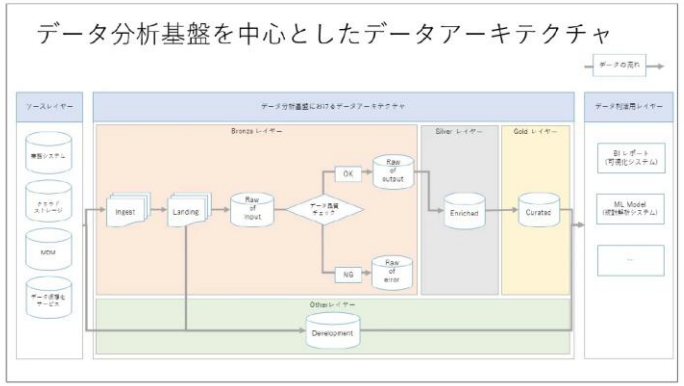
＜問題＞適切な開発方式や技術検証を持たずに進めること (1/2)

- システムのアーキテクチャの視点を欠いたまま開発を進めることはリスクが伴います。
- 特に Databricks を用いてシステムを構築する際には、
  構造化データ・非構造化データを問わず、データ管理の方法が非常に重要となります。

誰も教えてくれない
メダリオンアーキテクチャの デザインメソッド

2. メダリオンアーキテクチャの実装方法
2-1. 全体像

- メダリオンアーキテクチャとして、下記図のようなアーキテクチャを設計。
- 以前に関連する記事を投稿しており、現在は Ingest と Landing にて連携されたファイルをデータレイクで、それ以外を基本的にはマネージドテーブルで実装。

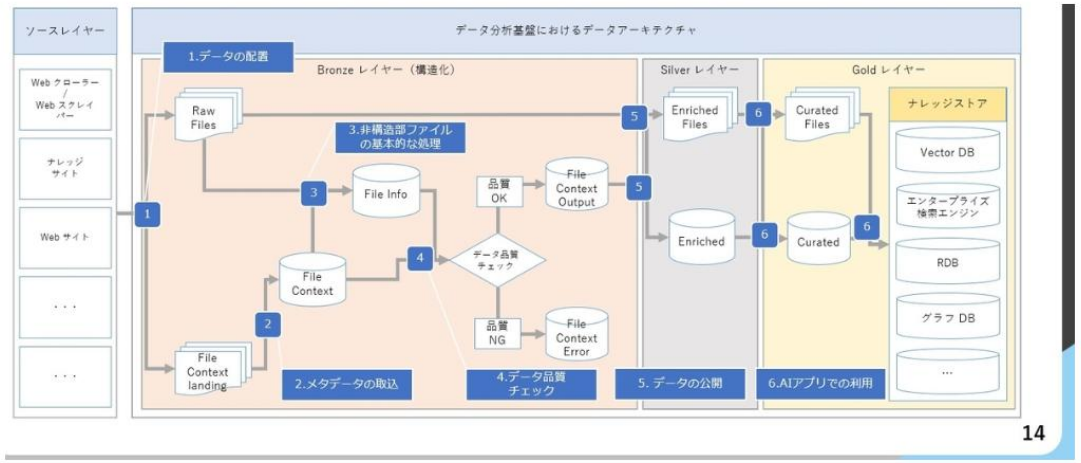


#	レイヤー	レイヤー詳細項目
0	Bronze	Ingest
1	Bronze	Landing
2	Bronze	Raw
3	Silver	Enriched
4	Gold	Curated
5	Other	Development

引用元：Databricks (Spark) における Spark テーブル（データレイク）のディレクトリ構成の検討 - Qiita

引用元：誰も教えてくれない メダリオンアーキテクチャの デザインメソッド | ドクセル

非構造化データのメダリオンアーキテクチャで
加速するAIアプリケーション開発



manabian様作のフロー

引用元：非構造化データのメダリオンアーキテクチャで加速するAIアプリケーション開発 - Vポイントマーケティング｜TECH LABの Tech Blog

<問題> 適切な開発方式や技術検証を持たずに進めること (2/2)

- 重要なオプションや動作仕様が十分に考慮されずにコードが実装されることがあります。
- ドキュメントの参照と動作確認による詳細な調査/検証の実施をおすすめします。

オプションが大量にあるため
ドキュメントの読み込む必要あり

汎用オプション

次のオプションは、すべてのファイル形式に適用されます。

オプション

`ignoreCorruptFiles` タイプ： `Boolean` 破損したファイルを無視するかどうか。 true の場合、破損したファイルに遭遇しても Spark ジョブは引き続き実行され、読み取られた内容は引き続き返されます。 で `numSkippedCorruptFiles` として観察可能 `operationMetrics` Delta Lake履歴の列。Databricks Runtime 11.3 LTS 以降で使用できます。 デフォルト値 `false`

`ignoreMissingFiles` タイプ： `Boolean` 見つからないファイルを無視するかどうか。 true の場合、不足しているファイルが検出されても Spark ジョブは引き続き実行され、読み取られた内容は引き続き返されます。 Databricks Runtime 11.3 LTS 以降で使用できます。 デフォルトの値: Auto Loaderは `false`、 `COPY INTO` (レガシ) は `true`

`modifiedAfter` タイプ： `Timestamp String` など `2021-01-01 00:00:00.000000 UTC+0` オプションのタイムスタンプは、指定されたタイムスタンプの後に変更タイムスタンプがあるファイルのみを取り込むためのフィルターとして。 デフォルト値: なし

引用元：Auto Loaderオプション | Databricks on AWS

発表時週に調査した
マルチテーブルトランザクション

@manabian

Databricks のマルチテーブルトランザクション機能の基本的な検証結果

Databricks

♡ 0

最終更新日 2026年04月16日 投稿日 2026年04月15日 84 views

概要

Databricks のマルチテーブルトランザクション機能の基本的な検証結果を共有します。

下記の処理を実行して、テーブルの履歴を確認する手順となっております。

1. 単一テーブルで INSERT を 3 回実行
2. 単一テーブルで INSERT + MERGE を実行
3. 単一テーブルで INSERT + DELETE を実行
4. 複数テーブルで INSERT + DELETE を実行

引用元：Databricks のマルチテーブルトランザクション機能の基本的な検証結果 #Databricks - Qiita

＜問題＞個人レベルでの活用方法にとどめてしまうこと(1/2)

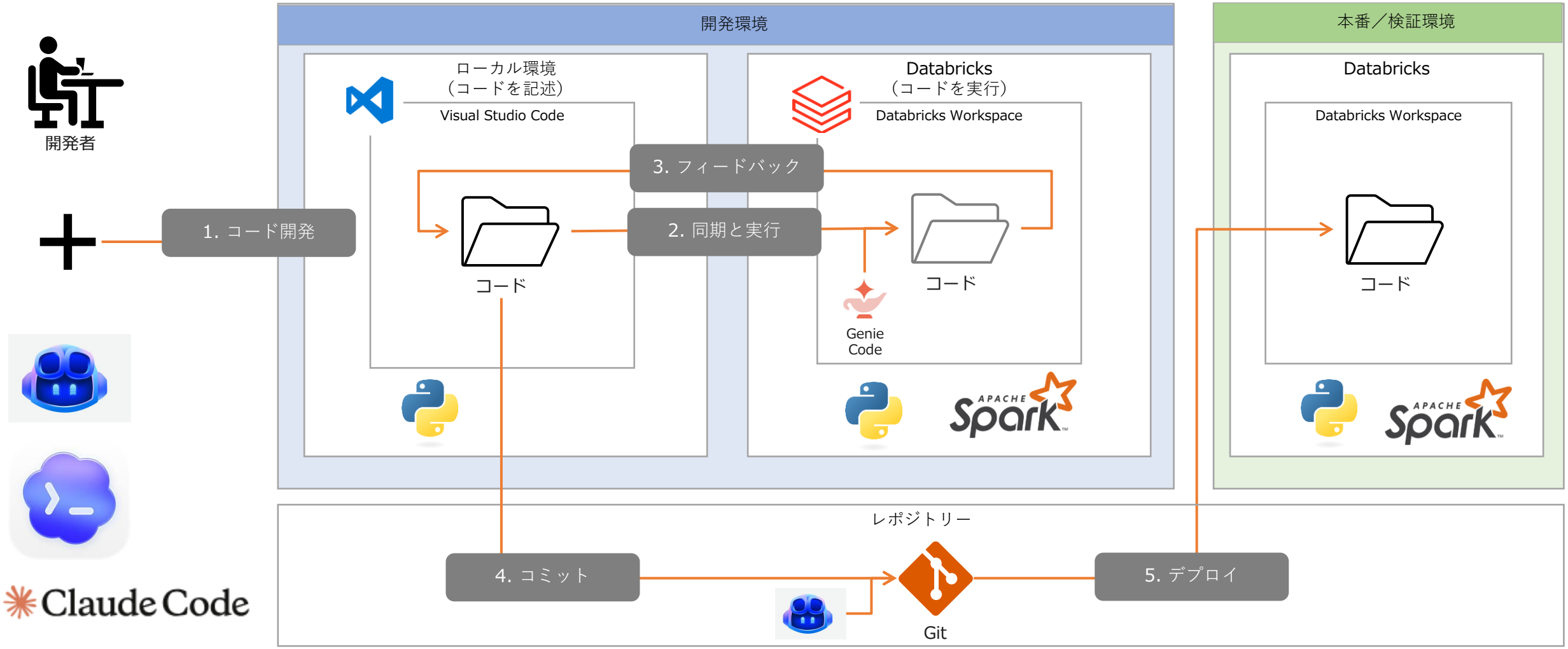
■ 個々のツール選定に注力するのではなく、高品質なコンテキスト（ナレッジ管理）を
いかに継続的に提供するかを検討する必要があります。

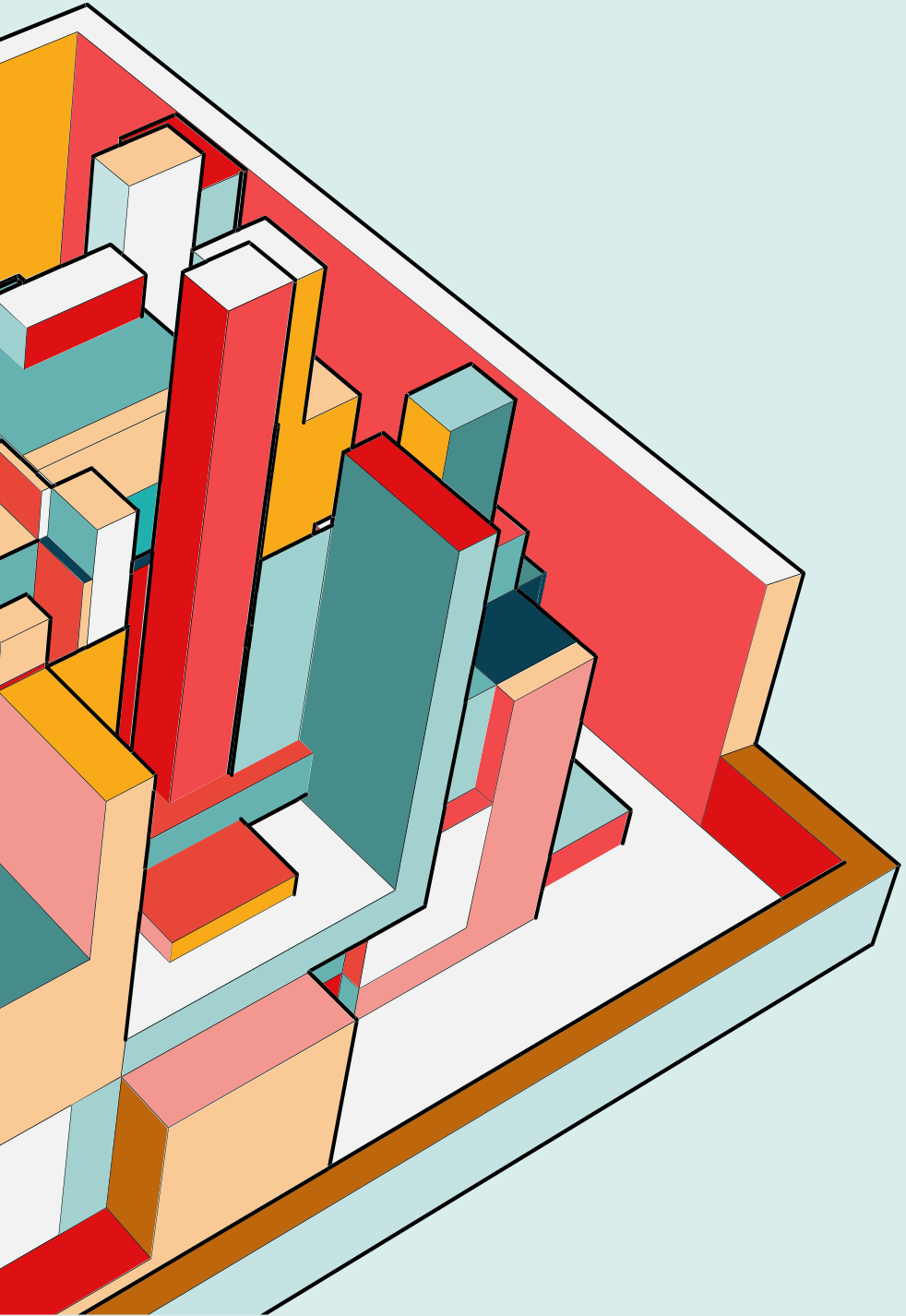
検討対象

ツール	コンテキスト（ナレッジ管理）
Claude Code	GitHub
	Jira

＜問題＞個人レベルでの活用方法にとどめてしまうこと(2/2)

■ ローカルでの開発方法論から含めて
開発ライフサイクルにおけるプロセスでどのように活用するかを検討する必要があります。





まとめ

まとめ

- 生成 AI は「ホームズ」、私たちは「ワトソン」— 情報を収集・判断し、ストーリー（システム）を構築する役割分担になってきています。
- AI の出力を鵜呑みにせず、組織（チーム）で拡張可能な方法論を一緒に考えましょう。

ありがとうございました

参考になった記事には
いいねをお願いします

manabian

𝕏: @manabian

Qiita: manabian